WATCHDOG 2.4

Portable Wireless Intrusion Detection System

Document Type	Technical Reference & Upgrade Guide
Firmware	`Watchdog_Final_V1.ino`
Version	v3.1 — Engineering Hardened
Architecture	Single-Loop, Async-Networks, Deferred-Action
Target MCU	ESP32 Dual-Core Xtensa LX6
RF Module	nRF24L01+ (125 channels, 2.400–2.524 GHz)
Display	ILI9341 TFT 320×240, SPI

Author: Anirudh N.S. — 2025–2026

Contents

1 Project Overview **3**

2 Hardware Architecture **3**
- 2.1 Microcontroller: ESP32 (Dual-Core Xtensa LX6) 3
- 2.2 RF Radio: nRF24L01+ 4
- 2.3 Display: ILI9341 TFT (320×240, SPI) 4
- 2.4 Touch Controller: XPT2046 4
- 2.5 Passive Buzzer 4
- 2.6 Battery ADC 4

3 Pin Map & Configuration Constants **5**
- 3.1 GPIO Assignment 5
- 3.2 Timing & Threshold Constants 6

4 Software Architecture & Design Philosophy **6**
- 4.1 Single-Loop, Asynchronous-Networks, Deferred-Action 6
- 4.2 The Deferred-Action Pattern 6
- 4.3 Memory Strategy 7
- 4.4 SPI Bus Sharing 7
- 4.5 FreeRTOS Mutex for BLE 7

5 Data Structures & Enumerations **7**
- 5.1 Enum: Page 7
- 5.2 Enum: JamType 8
- 5.3 Enum: PendingAction 8
- 5.4 Struct: WNet (Wi-Fi Network Entry) 8
- 5.5 Struct: BDev (Bluetooth/BLE Device Entry) 9
- 5.6 Struct: IAlert (Intrusion Alert Log Entry) 9

6 Global Variables Reference **9**
- 6.1 Core Detection Arrays (per-channel, 125 entries each) 9
- 6.2 Detection State Variables 10

7 System Initialisation (`setup`) **11**

8 Main Execution Loop (`loop`) **12**

9 Detection Engine — Core Algorithm **12**
- 9.1 Scale Multiplier 12
- 9.2 Per-Channel Processing 13
- 9.3 Shannon Entropy 13
- 9.4 Sweep Attack Detection 13
- 9.5 FHSS / Drone Link Detection 13
- 9.6 Signal Stability & Reflection Detection 14
- 9.7 Noise Floor Adaptation 14
- 9.8 History Counter Reset 14

10 Jam Classification Logic **14**

10.1 Instant Broadband Check (Pre-classification) . . . 14
10.2 Classification Waterfall . . . 15
10.3 Persistence Counters . . . 15
10.4 Jam Hold Behaviour . . . 15

11 RF Scanning Subsystem . . . 15

12 WiFi Scanning Subsystem . . . 16

13 BLE Scanning Subsystem . . . 16

14 OUI Database . . . 16
14.1 Camera OUI Table (`checkCameraOUI`) . . . 17
14.2 Tracker OUI & Name Table (`checkTrackerOUI`) . . . 17

15 Buzzer & Alarm Subsystem . . . 18

16 Power Management & Backlight . . . 18

17 User Interface — Page System . . . 19
17.1 Page Rendering Architecture . . . 19
17.2 Page-by-Page Reference . . . 19
17.2.1 Home Page (PG_HOME) . . . 19
17.2.2 RF Page (PG_RF) . . . 19
17.2.3 WiFi Page (PG_WIFI) . . . 19
17.2.4 BT Page (PG_BT) . . . 19
17.2.5 IDS Page (PG_IDS) . . . 19
17.2.6 SYS Page (PG_SYS) . . . 19

18 Touch Input Handling . . . 20

19 Function Quick Reference . . . 20

20 Upgrade Pathways & Future Features . . . 22
20.1 Adding a New Jam Type . . . 22
20.2 Expanding the OUI Database . . . 22
20.3 Adding Real Battery ADC . . . 22
20.4 Adding SD Card Logging . . . 23
20.5 Adding a New UI Page . . . 23
20.6 Persistent Storage via NVS Preferences . . . 23
20.7 Upgrading to a Rule Engine IDS . . . 23

21 Known Limitations & Bugs . . . 24

22 Version History . . . 25

1 Project Overview

WatchDog 2.4 is a handheld, battery-powered, portable wireless security analyser designed to detect, classify, and alert on jamming attacks, rogue devices, and anomalous RF behaviour across the full 2.4 GHz spectrum. It runs entirely offline on an ESP32 microcontroller with no cloud dependency, no external server, and no Wi-Fi connectivity requirement during operation.

Primary Mission: Give a field operator — or a TSCM (Technical Surveillance Countermeasures) technician — a device they can carry in a pocket that shows real-time RF spectrum activity, classifies threats automatically, identifies suspicious Bluetooth trackers, detects rogue cameras, and audibly alarms when jamming is confirmed.

Design Constraints

- Runs on a single ESP32 core for the main loop (Core 1); BLE callbacks run asynchronously on Core 0. The main loop must never block for long periods.
- All scanning operations (WiFi, BLE) are **non-blocking** (asynchronous state machines).
- The display and nRF24 share the same SPI bus; device selection is managed by manual CS toggling.
- Memory is severely constrained (≈320 KB heap shared with BLE stack). All strings use fixed-size `char` arrays — never Arduino `String` objects.
- Firmware must survive extended operation without heap fragmentation or stall crashes.

2 Hardware Architecture

2.1 Microcontroller: ESP32 (Dual-Core Xtensa LX6)

The ESP32 is the brain of the device. Key capabilities used:

- Dual-core processor (Core 0 runs BLE stack + callbacks; Core 1 runs the main sketch loop)
- Built-in Wi-Fi — used for scanning, not for connectivity
- Built-in Bluetooth Classic and BLE — used for device discovery
- SPI hardware peripheral — shared between TFT and nRF24
- LEDC peripheral — PWM-driven backlight control
- ADC1 channel 7 (GPIO 35) — battery voltage reading, safe alongside Wi-Fi
- Deep sleep with external wakeup via GPIO level

2.2 RF Radio: nRF24L01+

The nRF24L01 is a 2.4 GHz transceiver used purely as a **passive spectrum scanner**, not for bidirectional communication. It steps through all 125 channels (1 MHz apart, covering 2.400–2.524 GHz), reading the Received Power Detect (RPD) bit at each channel.

Parameter	Value & Rationale
AutoAck	Disabled — passive scanning only
CRC	Disabled — not transmitting
Address Width	2 bytes (minimum, speeds up reconfiguration)
Payload Size	32 bytes
Data Rate	RF24_2MBPS — minimises dwell time per channel
PA Level	RF24_PA_MIN — device is not transmitting; MIN prevents SPI self-interference

Table 1: nRF24L01+ configuration parameters

2.3 Display: ILI9341 TFT (320×240, SPI)

A 2.4-inch or 2.8-inch colour LCD driven over hardware SPI via the Adafruit_ILI9341 library. Rotated to landscape mode (`setRotation(1)`), giving a 320×240 pixel working area. A hardware reset sequence (RST line LOW for 80 ms, then HIGH) is performed at startup. Backlight brightness is controlled via the LEDC PWM peripheral on GPIO 27.

2.4 Touch Controller: XPT2046

A resistive touch controller on a separate SPI CS line. Raw ADC coordinates range from approximately 200–3800 on both axes and are mapped to pixel coordinates using calibration constants `TX_MIN`, `TX_MAX`, `TY_MIN`, `TY_MAX`.

2.5 Passive Buzzer

Connected to GPIO 26, driven using Arduino `tone()` and `noTone()` functions. Being passive, it plays any frequency, enabling distinct alarm melodies per jam type.

2.6 Battery ADC

GPIO 35 (ADC1_CH7) is prepared for battery voltage reading. ADC1 is safe alongside Wi-Fi (ADC2 cannot be used when Wi-Fi is active). In the current firmware, battery level is simulated via `virtualBattery`; the pin is defined and ready for a real voltage-divider implementation.

3 Pin Map & Configuration Constants

3.1 GPIO Assignment

Signal	GPIO	Notes
TFT_CS	5	TFT chip select (active LOW)
TFT_DC	32	TFT data/command select
TFT_RST	33	TFT hardware reset
TFT_BL_PIN	27	Backlight, PWM via LEDC
TOUCH_CS	13	XPT2046 chip select
TOUCH_IRQ	4	Touch interrupt; also deep sleep wakeup source
CE_PIN	22	nRF24L01 Chip Enable
CSN_PIN	21	nRF24L01 Chip Select Not
BUZZER_PIN	26	Passive buzzer
BAT_ADC_PIN	35	Battery ADC (ADC1_CH7)
Default SPI bus: MOSI=23, MISO=19, SCK=18 (shared by TFT, Touch, nRF24)		

Table 2: ESP32 GPIO assignment for WatchDog 2.4

3.2 Timing & Threshold Constants

Constant	Value	Purpose
TOTAL_CHANNELS	125	nRF24 channels 0–124
SCAN_PASSES	24	Full scan passes per detection cycle
EARLY_JAM_THRESHOLD	30	Active channels to trigger early detection
JAM_SPIKE_THRESHOLD	18	Instant broadband spike threshold
DETECT_THRESHOLD	1	Min above noise floor to count as active
SCALE_MULT_CAP	4.0f	Cap on early-pass energy scaling
DISPLAY_INTERVAL	120 ms	UI refresh rate
RF_INTERVAL	10 ms	RF scan chunk scheduling interval
IDLE_TIMEOUT	30000 ms	Time before deep sleep countdown
PEAK_HOLD_MS	1000 ms	Duration peak energy values are held
WIFI_SCAN_COOLDOWN	5000 ms	Minimum gap between WiFi scan triggers
BLE_SCAN_COOLDOWN	4000 ms	Minimum gap between BLE scan triggers
LONG_PRESS_MS	800 ms	Touch hold for mode cycling
TOUCH_DB	300 ms	Touch debounce period

Table 3: Timing and detection threshold constants

4 Software Architecture & Design Philosophy

4.1 Single-Loop, Asynchronous-Networks, Deferred-Action

The firmware follows a deliberately simple architecture that can be described in one sentence: **everything runs in a single cooperative loop except BLE callbacks, which run on a separate core and communicate through a mutex.**

This was chosen over an RTOS task-per-feature approach because:

- It is easier for AI models and humans to reason about sequentially
- There is no risk of priority inversion or deadlock except at the single BLE mutex boundary
- Display update logic is naturally serialised, preventing graphical corruption
- Loop timing issues are visible and debuggable via the health monitor

4.2 The Deferred-Action Pattern

Certain operations (theme toggle, data flush, noise calibration) cannot be safely performed inside the touch handler or during a display draw call, because they invalidate shared state mid-draw. These are queued as a `PendingAction` enum value and executed at the

start of the next loop iteration by `processPendingActions()`. This eliminates a class of rendering glitches and state corruption bugs.

4.3 Memory Strategy

All dynamic string operations were eliminated. Arduino `String` objects are never used. Every string field in structs (`WNet`, `BDev`, `IAlert`) uses a fixed `char` array with a hard null terminator written after every `strncpy`. This prevents heap fragmentation on the ESP32 — a real problem in long-running embedded applications.

4.4 SPI Bus Sharing

Three devices share the SPI bus. The rule is: assert only one CS line at a time. The TFT and nRF24 are never accessed simultaneously. During an RF scan chunk, the nRF24 CS is active. During display drawing, the TFT CS is active. The touch CS is asserted only when `handleTouch()` polls the XPT2046.

4.5 FreeRTOS Mutex for BLE

BLE Thread Safety

BLE callbacks (`BLECb::onResult`) run on **Core 0**, allocated by the ESP-IDF BLE stack. The main loop runs on **Core 1**. The shared data region is: `bDevs[]`, `bCount2`, `bSus`, `bTrackers`.
A FreeRTOS binary semaphore (`bleMutex`) protects this region. Every read in the main loop takes the mutex with a 5–10 ms timeout; every write in the BLE callback takes it with a 10 ms timeout. The mutex is created in `setup()` *before* any BLE initialisation.

5 Data Structures & Enumerations

5.1 Enum: Page

```
enum Page {
  PG_HOME,    // Dashboard: threat level, overview stats
  PG_RF,      // 2.4 GHz spectrum analyser view
  PG_WIFI,    // Wi-Fi network mapper, channel display, foxhunt
  PG_BT,      // Bluetooth/BLE device mapper, tracker detection
  PG_IDS,     // Wireless IDS log and persistence counters
  PG_SYS,     // System diagnostics, uptime, calibration, theme
  PG_COUNT    // Sentinel = 6, used for modulo cycling
};
```

The nav bar at the bottom renders one labelled button per page. Touching any nav zone navigates directly to that page. Touching the main content area cycles to the next page modulo `PG_COUNT`.

5.2 Enum: JamType

```
enum JamType {
  JAM_NONE,          // No threat detected
  JAM_RC,            // RC/drone remote control (channels 84-124)
  JAM_BT,            // Bluetooth Classic (40 channels)
  JAM_BLE,           // Bluetooth Low Energy (40 even-numbered
      channels)
  JAM_BT_WIFI,       // Combined BT + WiFi interference pattern
  JAM_WIFI,          // Wi-Fi channels 1-14 (nRF24 channels 0-83)
  JAM_WIFI_DEAUTH,   // WiFi deauthentication flood attack
  JAM_DRONE_FHSS,    // Frequency-hopping spread-spectrum (drone
      control)
  JAM_SWEEP,         // Linear sweep jammer (peak moving across
      spectrum)
  JAM_ANOMALY,       // High Shannon entropy, anomalous RF environment
  JAM_BROADBAND      // Broadband jammer (all channels simultaneously)
};
```

Classification priority runs from most specific (broadband instant) down to most generic. Once a `JamType` is assigned, the buzzer melody, display label, and IDS alert are all derived from it.

5.3 Enum: PendingAction

```
enum PendingAction {
  ACT_NONE,          // No deferred action queued
  ACT_THEME_TOGGLE,  // Invert display theme on next loop
  ACT_ZERO_NOISE,    // Capture current noise floor as baseline
  ACT_BLE_FLUSH,     // Clear BLE device list and restart scan
  ACT_WIFI_FLUSH,    // Clear Wi-Fi network list
  ACT_IDS_FLUSH      // Clear alert log
};
```

5.4 Struct: WNet (Wi-Fi Network Entry)

```
struct WNet {
  char ssid[33];   // Network name, null-terminated, max 32 chars
  char mac[18];    // BSSID as "XX:XX:XX:XX:XX:XX\0"
  int  rssi;       // Signal strength in dBm (typically -30 to -90)
  int  ch;         // Wi-Fi channel (1-14)
  bool open;       // True if WIFI_AUTH_OPEN
  bool twin;       // True if another net with same SSID found (Evil
      Twin)
  bool isCam;      // True if MAC OUI matches known IP camera vendor
};
// Max 20 entries in global wNets[] array
```

5.5 Struct: BDev (Bluetooth/BLE Device Entry)

```
struct BDev {
  char name[32];     // Device name or MAC address string, null-terminated
  char mac[18];      // MAC as "XX:XX:XX:XX:XX:XX\0"
  int  rssi;         // Signal strength in dBm
  bool sus;          // True if no name AND RSSI > -65 (suspicious hidden device)
  bool isTracker;    // True if MAC OUI or name matches known tracker
};
// Max 20 entries in global bDevs[] array     protected by bleMutex
```

5.6 Struct: IAlert (Intrusion Alert Log Entry)

```
struct IAlert {
  char type[20];       // Short type label e.g. "WiFi_Deauth_Flood"
  char detail[28];     // MAC address, SSID, or "RF Threat"
  unsigned long ts;    // Timestamp in seconds since boot (millis()/1000)
};
// Circular buffer of 5 entries in global alerts[]
// Most recent is always at index 0; entries at index > 4 discarded
```

6 Global Variables Reference

6.1 Core Detection Arrays (per-channel, 125 entries each)

Variable	Type	Purpose
`energy[]`	`uint8_t[125]`	Raw energy reading from current scan pass
`smoothEnergy[]`	`uint8_t[125]`	Exponentially smoothed energy; used for display and detection
`prevEnergy[]`	`uint8_t[125]`	Previous smooth value for delta-rendering on RF page
`peakEnergy[]`	`uint8_t[125]`	Peak-hold value per channel
`prevPeak[]`	`uint8_t[125]`	Previous peak value for delta-rendering
`noiseFloor[]`	`float[125]`	Adaptive noise floor baseline per channel
`peakTime[]`	`ulong[125]`	Timestamp of last peak update for decay logic

Table 4: Per-channel detection arrays

6.2 Detection State Variables

Variable	Type	Init	Purpose
`scanCh`	int	0	Current channel being scanned
`scanPass`	int	0	Current pass within a 24-pass cycle
`earlyActive`	int	0	Active channels in current pass
`instantSpike`	int	0	Broadband spike detection counter
`scanDone`	bool	false	True when full 24-pass cycle complete
`isInstantJam`	bool	false	Set when JAM_SPIKE_THRESHOLD crossed
`activeCount`	int	0	Channels above noise floor
`activePct`	float	0	activeCount as % of 125
`wifiCount`	int	0	Peak contiguous active channels (WiFi sig.)
`btCount`	int	0	Non-RC active channels (BT signature)
`bleCount`	int	0	Even-numbered active channels (BLE sig.)
`rcCount`	int	0	Active channels 84–124 (RC signature)
`wifiHist`	int	0	Persistence counter for WiFi classification
`btHist`	int	0	Persistence counter for BT classification
`bleHist`	int	0	Persistence counter for BLE classification
`rcHist`	int	0	Persistence counter for RC classification
`anomalyHist`	int	0	Persistence counter for entropy anomaly
`sweepHist`	int	0	Persistence counter for sweep attack
`hopCounter`	int	0	Large peak-channel jumps within 2 s
`hopDistanceSum`	int	0	Sum of channel distances for FHSS

Variable	Type	Init	Purpose
`sweepCount`	int	0	Consecutive directional peak movements
`sweepDirection`	int	0	+1 (upward) or -1 (downward) sweep
`stableSignal`	bool	false	Variance < 4 and avg > 8
`jamming`	bool	false	Active jam classification confirmed
`jamType`	JamType	NONE	Current active jam type
`heldJamType`	JamType	NONE	Last confirmed type, held 1 s after drop

Table 5: Detection state variables

7 System Initialisation (`setup`)

The `setup()` function runs once at power-on or after deep sleep wake. It follows a strict ordering to prevent hardware conflicts.

1. **BLE Mutex Creation** — `bleMutex` is created as the very first action. This ensures the mutex exists before any BLE stack activity.
2. **Buzzer & Backlight** — Buzzer pin set to OUTPUT and silenced. LEDC PWM configured with a compile-time API version check (ESP Arduino 3.x vs. 2.x fallback).
3. **SPI CS Line Defaults** — All CS lines (TFT_CS, TOUCH_CS, CSN_PIN) pulled HIGH (deasserted). CE_PIN pulled LOW.
4. **TFT Hardware Reset** — TFT_RST held LOW for 80 ms then HIGH for 80 ms.
5. **Display & Touch Init** — `tft.begin()` and `touch.begin()`, both set to landscape rotation 1.
6. **Splash Screen** — If `wakeFromSleep` is false, a branded splash screen with animated progress bar is displayed (≈1 s).
7. **nRF24 Initialisation** — `radio.begin()` called; on failure a permanent error screen halts execution. Radio configured for passive scanning.
8. **BLE Init** — `BLEDevice::init("WATCHDOG")` starts BLE stack. Active scanning enabled to request scan response packets (extended device names).
9. **Array Initialisation** — `smoothEnergy`, `energy`, `noiseFloor` zeroed. Timestamps seeded with `millis()`.
10. **First Draw** — `applyTheme()` sets colour variables, `needRedraw = true`, `redrawPage()` renders Home page.

8 Main Execution Loop (`loop`)

The `loop()` function is the cooperative scheduler. It runs as fast as possible (no `delay()`) and calls each subsystem in a defined order every iteration.

#	Call	Purpose
1	Buzzer-off check	Silence buzzer if `buzzerOffTime` elapsed
2	`handleTouch()`	Read and debounce touch input
3	`processPendingActions()`	Execute any queued deferred action
4	`buzzerUpdate()`	Advance alarm melody state
5	`handlePowerAndBacklight()`	Battery drain, dim/wake screen, idle timeout
6	`runGeigerCounter()`	Emit proximity beeps in foxhunt modes
7	Popup auto-dismiss	Dismiss popup after 1800 ms
8	`runRFChunk(now)`	One RF scan step if interval elapsed
9	`scanWifiAsync()`	Poll Wi-Fi scan state machine
10	BLE re-trigger check	Start new BLE scan if cooldown elapsed
11	`runHealthMonitor()`	Measure loop timing, flag stalls
12	Display update	`redrawPage()` if `needRedraw` and interval elapsed

Table 6: Main loop execution order

The only calls that can exceed a few microseconds are TFT drawing calls (2–20 ms per partial fill depending on area). Everything else is non-blocking.

9 Detection Engine — Core Algorithm

`runDetection()` is called at the end of each complete 24-pass RF scan cycle. It is the most complex function in the firmware.

9.1 Scale Multiplier

Early-trigger can call `runDetection()` with fewer than 24 passes. A `scaleMultiplier` compensates by inflating raw energy readings:

$$\text{scaleMultiplier} = \min\left(\frac{\texttt{SCAN_PASSES}}{\text{actualPasses}},\ \texttt{SCALE_MULT_CAP}\right) = \min\left(\frac{24}{\text{actual}},\ 4.0\right)$$

The cap prevents a single-pass early trigger from saturating the smoothed energy buffer.

9.2 Per-Channel Processing

For each of the 125 channels:

Asymmetric Smoothing:

$$\text{smoothEnergy}[ch] = \begin{cases} \text{scaledE} & \text{if scaledE} > \text{smoothEnergy}[ch] \quad \text{(fast attack} \\ \dfrac{\text{smoothEnergy}[ch] \times 2 + \text{scaledE}}{3} & \text{otherwise} \quad \text{(slow decay)} \\ \lfloor \text{smoothEnergy}[ch]/2 \rfloor & \text{if scaledE} = 0 \\ 0 & \text{if smoothEnergy}[ch] \leq 3 \quad \text{(clamp to zero)} \end{cases}$$

Activity Buckets: A channel is "active" if $\text{smoothEnergy}[ch] \geq \text{noiseFloor}[ch] + \texttt{DETECT_THRESHOLD}$. Active channels sort into:

- `rcHits` — channels 84–124 (RC/drone band)
- `evenHits` — even-numbered channels below 84 (BLE channels)
- `oddHits` — odd channels below 84
- `nonRcActive` — all below-84 active channels (Bluetooth region)

Anomaly Scoring: Any channel with deviation > 3 above its noise floor contributes its deviation to `anomalyScore`.

9.3 Shannon Entropy

$$H = -\sum_{i=0}^{124} p_i \log(p_i), \quad p_i = \frac{\text{smoothEnergy}[i]}{\sum_j \text{smoothEnergy}[j]}$$

`highEntropy` is true if:

- $H > 3.8$ and `activeCount` > 20, **or**
- `anomalyScore` > 200 and `activeCount` < 15 (burst transmitter pattern)

9.4 Sweep Attack Detection

The peak energy channel is tracked across detection cycles. A sweep is confirmed (`isSweepAttack`) when:

- The peak channel moves 4–15 channels per cycle (jumps > 15 reset the counter)
- Movement is consistently in the same direction (`sweepDirection`)
- This pattern continues for more than 6 consecutive cycles (`sweepCount` > 6)

9.5 FHSS / Drone Link Detection

Within a 2-second rolling window, large peak channel jumps (> 15 channels) are accumulated. The pattern is classified as FHSS (drone control link) when:

$$\texttt{hopCounter} \geq 5 \quad \text{AND} \quad \texttt{hopDistanceSum} \geq 80$$

This implies an average hop of > 16 channels (> 16 MHz per hop), consistent with drone control frequency hopping profiles.

9.6 Signal Stability & Reflection Detection

An 8-sample ring buffer (`stabilityBuffer`) holds recent maximum energy values. The signal is `stableSignal` when variance < 4 and average > 8. A `reflectionSpike` counter increments when the signal is stable but shows an instantaneous delta > 8 units (multipath reflection signature). `reflectionSpike` is capped at 20.

9.7 Noise Floor Adaptation

When $< 5\%$ of channels are active and maximum energy < 5 (quiet environment):

$$\text{noiseFloor}[ch] = \frac{\text{noiseFloor}[ch] \times 63.0 + \text{smoothEnergy}[ch]}{64.0}$$

This 64-sample EMA prevents false positives after relocation to a denser RF environment.

9.8 History Counter Reset

If `activeCount` < 3 and no jam detected in the last 1000 ms, all six persistence counters (`wifiHist`, `btHist`, `bleHist`, `rcHist`, `anomalyHist`, `sweepHist`) are reset to zero. This prevents stale history from causing false classification after a jammer stops.

10 Jam Classification Logic

10.1 Instant Broadband Check (Pre-classification)

During the RF scan pass loop, if `earlyActive` reaches `EARLY_JAM_THRESHOLD = 30` before 24 passes complete AND channel energy exceeds `JAM_SPIKE_THRESHOLD = 18`, `isInstantJam` is set and `runDetection()` is called immediately. This reduces broadband detection latency from ≈2.5 s to ≈0.5 s.

10.2 Classification Waterfall

Priority	Type	Condition
1	JAM_BROADBAND	`isInstantJam` is true
2	JAM_WIFI_DEAUTH	`wifiCount` ≥ 10 AND `wifiHist` ≥ 3 AND `activePct` ≥ 25 AND `btCount` < 20
3	JAM_DRONE_FHSS	Not above; `hopCounter` ≥ 5, `hopDistanceSum` ≥ 80, `activePct` < 40
4	JAM_SWEEP	Not above; `sweepHist` ≥ 2
5	JAM_ANOMALY	Not above; `anomalyHist` > 2
6	JAM_RC	Not above; `rcCount` ≥ 3, `activePct` > 30
7	JAM_WIFI	Not above; `wifiCount` ≥ 10, peak energy at `bestWiFiCenter` elevated
8	JAM_BLE	Not above; `bleCount` ≥ 10, `oddHits` $\leq$ `evenHits/2 + 4`
9	JAM_BT	Not above; `btCount` ≥ 15

Table 7: Jam type classification waterfall (evaluated in priority order)

10.3 Persistence Counters

Each classification requires its condition to be true across multiple consecutive detection cycles. History counters increment when true and reset to zero when false. Most types require a history value of 2 (true in 2 consecutive cycles) before `jamming` is set. This eliminates false positives from burst transmitters or isolated interference spikes.

10.4 Jam Hold Behaviour

When detection signal disappears (`newType == JAM_NONE`), `jamming` remains true and `jamType` shows `heldJamType` for up to 1000 ms. This prevents display flickering when a jammer briefly dips below threshold.

11 RF Scanning Subsystem

`runRFChunk(now)` is called every `RF_INTERVAL = 10 ms` from the main loop. It executes one atomic RF scan operation per call.

Pass structure: Each complete scan cycle has 24 passes. In each pass, all 125 channels are visited. The nRF24 is tuned to each channel, a brief dwell allows the RPD register to settle, and the RPD bit is read. If RPD is high, `energy[scanCh]` is incremented.

Early trigger: If `earlyActive` reaches `EARLY_JAM_THRESHOLD` during a pass AND the energy at the current channel exceeds `JAM_SPIKE_THRESHOLD`, `isInstantJam` is set and `scanDone` is forced. `earlyActive` is reset to zero at the start of each pass to prevent a single-pass false positive from carrying over.

Scan completion: When `scanPass` reaches `SCAN_PASSES = 24`, `runDetection()` is

called, `energy[]` is zeroed, and `scanPass` resets to 0.

12 WiFi Scanning Subsystem

`scanWifiAsync()` is a state machine called each loop iteration when the user is on the WiFi page. It uses the ESP32 Arduino WiFi library's non-blocking scan API.

State machine: If no scan is in progress and the cooldown has elapsed, a background scan is started with `WiFi.scanNetworks(true, true)` (async=true, show_hidden=true). `WiFi.scanComplete()` is polled on subsequent calls. When it returns a non-negative count, results are read into `wNets[]`. Scan result memory is freed with `WiFi.scanDelete()`.

Evil Twin detection: For every pair of networks with identical SSIDs, both entries have their `twin` flag set to true and an "Evil Twin" alert is added to the IDS log.

Camera detection: Each MAC is checked against the camera OUI table. If matched, `isCam` is set and a "Camera Found" alert is added.

13 BLE Scanning Subsystem

`triggerAsyncBLE()` takes the mutex, clears the BLE device arrays, releases the mutex, then starts a 1-second asynchronous BLE scan with `bleScan->start(1, bleScanCompleteCB, false)`.

`BLECb::onResult()` is the callback invoked on Core 0 by the BLE stack for each advertisement received during the scan window. It takes the mutex (10 ms timeout), fills a `bDev` entry, runs tracker and suspicious-device detection, then releases the mutex.

Active scanning: `setActiveScan(true)` causes the ESP32 to send scan request packets. Devices respond with scan response packets that often contain the full device name. This significantly improves device identification.

Suspicious device heuristic: A device is flagged `sus = true` if it has no name (advertising anonymously) AND its RSSI is stronger than −65 dBm (close range). This is consistent with covert tracking device behaviour.

14 OUI Database

14.1 Camera OUI Table (`checkCameraOUI`)

OUI Prefix	Vendor	Notes
`EC:FA:BC`	Hikvision	Major IP camera manufacturer
`AC:64:62`	Hikvision	Alternate OUI block
`44:65:0D`	Dahua	Major IP camera manufacturer
`70:8A:09`	Reolink	Consumer IP cameras
`9C:8E:CD`	Wyze	Low-cost smart cameras
`2C:AA:8E`	TP-Link Tapo	Smart home cameras
`50:C7:BF`	Ezviz	Hikvision consumer brand

Table 8: Camera OUI lookup table

14.2 Tracker OUI & Name Table (`checkTrackerOUI`)

OUI / Keyword	Vendor/Product	Method
`00:25:96`	Apple AirTag	OUI match
`7C:D1:C3`	Tile	OUI match
`smarttag`	Samsung	Case-insensitive name search
`airtag`	Apple	Case-insensitive name search
`findmy`	Apple	Case-insensitive name search
`tile`	Tile	Case-insensitive name search
`chipolo`	Chipolo	Case-insensitive name search

Table 9: Tracker detection: OUI prefixes and name keywords

The name search uses a `char lower[32]` buffer with a manual `tolower` loop — no heap allocation.

15 Buzzer & Alarm Subsystem

`buzzerStartAlarm(JamType j)` sets `alarmActive = true`, records the jam type, and resets phase and repeat counters. `buzzerStopAlarm()` clears `alarmActive` and calls `noTone()`.

`buzzerUpdate()` is called every loop iteration. Alarm interval is threat-severity-based:

activePct	Interval	Frequency pair
$\geq 70\%$	150 ms	Deauth/Drone/Sweep: 3800 Hz / 4200 Hz (high urgency)
$\geq 45\%$	300 ms	Others: 2730 Hz / 4000 Hz
$\geq 25\%$	600 ms	
$< 25\%$	1000 ms	

Table 10: Alarm intervals and frequencies by threat severity

Geiger Counter Mode: `runGeigerCounter()` produces clicking sounds proportional to signal strength in Foxhunt modes. Click interval is mapped from signal strength: -90 dBm $\rightarrow$ 1200 ms interval; -30 dBm $\rightarrow$ 40 ms interval.

16 Power Management & Backlight

Virtual Battery: `virtualBattery` starts at 100.0% and drains every 5000 ms. Normal mode simulates ≈65 minutes; ECO mode simulates ≈120 minutes.

ECO Mode: Toggled by a 5-second long press. When active: backlight dimmed to 75/255 (≈30%), BLE scanning paused, battery drain reduced by ≈45%.

Idle Timeout & Deep Sleep: After `IDLE_TIMEOUT = 30 s` with no touch input, a 10-second countdown is displayed. If countdown expires:

1. `wakeFromSleep = true` is set in RTC memory (survives deep sleep)
2. TOUCH_IRQ (GPIO 4) configured as wakeup source (level LOW)
3. `esp_deep_sleep_start()` is called

On wake, `wakeFromSleep = true` ensures the splash screen is skipped.

17 User Interface — Page System

17.1 Page Rendering Architecture

`redrawPage()` detects page transitions via a `lastDrawnPage` static variable. On page change, the screen is cleared and the nav bar redrawn. The RF page uses differential rendering: `prevEnergy[]` and `prevPeak[]` store previously drawn values; only changed channels are repainted, reducing TFT draw calls by ≈80–90% in typical RF environments.

17.2 Page-by-Page Reference

17.2.1 Home Page (PG_HOME)

Displays: global threat level (LOW/MEDIUM/HIGH/CRITICAL), active channel count and percentage, threat level progress bar, detection counters (WiFi/BT/BLE/RC). `homeMode` cycles audible/silent.

17.2.2 RF Page (PG_RF)

`rfMode` cycles through 5 sub-modes:

1. **Standard Spectrum** — all 125 channels as a bar chart
2. **Zoom Lo** — channels 0–62 (Wi-Fi/BT band detail)
3. **Zoom Hi** — channels 63–124 (RC/drone band detail)
4. **Peak Freeze** — smoothed energy with peak hold lines locked
5. **Foxhunt** — single proximity bar for strongest channel with Geiger clicking

17.2.3 WiFi Page (PG_WIFI)

`wifiMode` cycles through 5 sub-modes:

1. **Network List** — SSID, MAC, RSSI, channel, flags (open/camera/twin)
2. **Channel Usage** — 14 Wi-Fi channels shown as occupancy bars
3. **Threat Map** — colour-coded channel danger summary
4. **Foxhunt** — strongest RSSI as proximity bar
5. **Deep Flush** — clears Wi-Fi data (deferred action)

17.2.4 BT Page (PG_BT)

`btMode` cycles through 4 sub-modes. *All draws take a mutex snapshot of `bDevs[]` into `localDevs[]` before rendering, to prevent race conditions with BLE callbacks.*

1. **All Devices** — name/MAC, RSSI, 5-bar signal strength indicator
2. **Trackers Only** — filtered view of flagged tracker devices (highlighted in red)
3. **Foxhunt** — strongest RSSI device as proximity bar
4. **Deep Flush** — clears BLE data

17.2.5 IDS Page (PG_IDS)

Displays: channel activity counters with colour-coded thresholds, five persistence counter bar charts (wifiHist, btHist, sweepHist, rcHist, anomalyHist) with ALERT labels at threshold, current alarm status, and the last three IDS alert entries with ages in seconds.

17.2.6 SYS Page (PG_SYS)

Displays: uptime, active channel %, jamming status, attack type, alarm state, total alerts, Wi-Fi network count, BLE device count, max loop delay (red > 2 s), stall recovery count,

theme name, firmware version.

18 Touch Input Handling

Coordinate Mapping:

$$x_{\text{pixel}} = \text{map}(\text{rawX},\ \texttt{TX_MIN},\ \texttt{TX_MAX},\ 0,\ 320)$$

$$y_{\text{pixel}} = \text{map}(\text{rawY},\ \texttt{TY_MIN},\ \texttt{TY_MAX},\ 0,\ 240)$$

Debounce: 300 ms (`TOUCH_DB`) prevents multiple triggers from a single press.

Short Press (< 800 ms): On release, navigates if tap in bottom 28 px (nav bar); otherwise cycles to next page. Also dismisses active alarm.

Long Press (800 ms): Triggers mode cycling within the current page. Each mode change fires a popup with the mode name.

Very Long Press (5000 ms): Toggles ECO mode on/off.

19 Function Quick Reference

Function	Section	Purpose
`setup()`	Init	Hardware, splash, radio, BLE init
`loop()`	Main Loop	Cooperative task scheduler
`runDetection()`	Detection	Classify RF threats from scan data
`runRFChunk(now)`	RF Scanner	One step of nRF24 scan
`scanWifiAsync()`	WiFi	Non-blocking WiFi scan state machine
`triggerAsyncBLE()`	BLE	Start a 1-second BLE scan
`BLECb::onResult()`	BLE Callback	Per-device handler (Core 0)
`handleTouch()`	Touch	Read, debounce, decode touch events
`processPendingActions()`	Deferred	Execute queued state changes
`buzzerUpdate()`	Alarm	Advance alarm melody state
`buzzerStartAlarm(j)`	Alarm	Begin alarming for jam type j
`buzzerStopAlarm()`	Alarm	Silence alarm
`triggerBeep(dur, freq)`	Alarm	Fire one-shot beep
`runGeigerCounter()`	Geiger	Proximity clicking in foxhunt modes
`handlePowerAndBacklight()`	Power	Battery, dim/wake, deep sleep

Function	Section	Purpose
`applyTheme()`	Theme	Set global colour variables
`drawStatusBar()`	UI	Battery and ECO indicator
`redrawPage()`	UI	Full page redraw dispatcher
`drawNav(p)`	UI	Render bottom navigation bar
`drawHome()`	UI	Render home/dashboard page
`drawRF_Static()`	UI	Render static RF page elements
`drawRF_Dynamic()`	UI	Render RF bar chart (full)
`updateRFBars()`	UI	Differential RF bar update
`drawWifi()`	UI	Render WiFi page (all modes)
`drawBT()`	UI	Render BT page (all modes)
`drawIDS()`	UI	Render IDS page
`drawSys()`	UI	Render SYS diagnostics
`triggerPopup(...)`	Popup	Set and show popup notification
`drawPopup()`	Popup	Render popup overlay
`addAlert(type, detail)`	IDS	Add entry to alert log
`jamLabel(j)`	Utility	Return string label for JamType
`checkCameraOUI(mac)`	OUI	True if MAC matches camera vendor
`checkTrackerOUI(mac, name)`	OUI	True if MAC/name matches tracker
`drawCenteredText(...)`	UI	Render centred text at given y
`runHealthMonitor()`	Health	Measure loop timing, flag stalls

Table 11: Function quick reference

20 Upgrade Pathways & Future Features

AI Re-entry Note

This section is written specifically so that AI models (Claude/Gemini) and engineers re-entering the project can add new capabilities without breaking the existing architecture. Each subsection provides precise, step-by-step instructions.

20.1 Adding a New Jam Type

1. Add enum value to `enum JamType` (add at the end, before the closing brace)
2. Add detection condition in `runDetection()` using the history-counter pattern (new `xxxHist` variable, increment/reset logic)
3. Add `int xxxHist = 0;` to global variables
4. Add case to the classification waterfall in the `JamType newType =` block
5. Add case to `jamLabel()` returning the string label
6. Optionally add unique buzzer melody pattern to `buzzerUpdate()` switch
7. Add IDS page display if a persistence bar is needed in `drawIDS()`

20.2 Expanding the OUI Database

The `checkCameraOUI()` and `checkTrackerOUI()` functions use linear `strncmp` chains. For > 20 entries, refactor to a sorted array with `bsearch()`, or store in PROGMEM:

```
// Example PROGMEM OUI table structure
struct OUIEntry { const char oui[9]; const char vendor[20]; };
const OUIEntry CAMERA_OUIS[] PROGMEM = {
  {"EC:FA:BC", "Hikvision"},
  {"AC:64:62", "Hikvision"},
  // ... add more entries here
};
```

20.3 Adding Real Battery ADC

Replace the `virtualBattery` drain simulation in `handlePowerAndBacklight()` with:

```
// In handlePowerAndBacklight():
esp_adc_cal_characteristics_t adc_chars;
esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_11,
                         ADC_WIDTH_BIT_12, 1100, &adc_chars);
uint32_t raw = analogRead(BAT_ADC_PIN);
uint32_t voltage_mv = esp_adc_cal_raw_to_voltage(raw, &adc_chars) *
    2; // *2 for divider
// Map voltage_mv to percentage using LiPo discharge curve
virtualBattery = map(constrain(voltage_mv, 3200, 4200), 3200, 4200,
    0, 100);
```

20.4 Adding SD Card Logging

An SD card (SPI) can be added on a new CS pin. The SPI bus sharing model already in use supports a fourth device with no architecture changes:

1. Define `#define SD_CS_PIN 15` (or any free GPIO)
2. Include `#include <SD.h>` and call `SD.begin(SD_CS_PIN)` in `setup()`
3. In `addAlert()`, after updating the `alerts[]` ring buffer, open the log file and append a CSV row
4. In `runDetection()`, log detection events with timestamp, type, activePct, and entropy

20.5 Adding a New UI Page

1. Add enum value to `enum Page` before `PG_COUNT`
2. Add label and colour entries to `drawNav()` arrays `L[]` and `C[]`
3. Write `void drawMyPage()` rendering function
4. Add `case PG_MYPAGE: drawMyPage(); break;` in `redrawPage()`
5. Add touch mode cycling in `handleTouch()` if the page has sub-modes

20.6 Persistent Storage via NVS Preferences

Noise floor calibration and user settings are lost on power cycle. Use the ESP32 NVS:

```
#include <Preferences.h>
Preferences prefs;

// Save noise floor (125 floats = 500 bytes, well within NVS)
void saveNoiseFloor() {
  prefs.begin("watchdog", false);
  prefs.putBytes("nf", noiseFloor, sizeof(noiseFloor));
  prefs.putBool("theme", isLightMode);
  prefs.putBool("mute", homeMode == 1);
  prefs.end();
}

// Load on startup (call after arrays are initialised)
void loadPreferences() {
  prefs.begin("watchdog", true);
  prefs.getBytes("nf", noiseFloor, sizeof(noiseFloor));
  isLightMode = prefs.getBool("theme", false);
  homeMode = prefs.getBool("mute", false) ? 1 : 0;
  prefs.end();
}
```

20.7 Upgrading to a Rule Engine IDS

Replace the hard-coded classification waterfall with a struct-driven rule engine:

```
struct DetectionContext {
  int activeCount; float activePct;
  int wifiCount, btCount, bleCount, rcCount;
  int hopCounter, hopDistanceSum, sweepCount;
  bool highEntropy, isInstantJam;
};

```

```
struct IDSRule {
  const char* name;
  bool (*evaluate)(const DetectionContext& ctx);
  JamType resultType;
  int historyRequired;
  int* histCounter;  // pointer to the rule's global history counter
};

// Define rules as a const array     easy to add/reorder/disable
const IDSRule RULES[] = {
  {"Broadband", [](const DetectionContext& c){ return c.isInstantJam;
      },
   JAM_BROADBAND, 1, &rcHist /* not used */},
  // ... add more rules
};
```

21 Known Limitations & Bugs

Known Issues — v3.1

1. **Virtual Battery Only** — `BAT_ADC_PIN` is defined but not read. Battery level is purely time-simulated. A voltage divider and ADC calibration are required for real hardware.
2. **BLE Scan Array Cleared on Re-trigger** — `triggerAsyncBLE()` clears `bCount2` under the mutex before starting a new scan. If the previous scan is still in progress, partial results are lost. Solution: double-buffer `bDevs[]`.
3. **nRF24 Channel Dwell Time** — RPD bit is read once per channel with no explicit dwell delay. Very fast FHSS signals (dwell $< 130\,\mu$s) may be missed.
4. **Evil Twin by SSID Only** — Detection compares SSID strings only. A homoglyph or extra-space attack would not be caught. Future improvement: also compare channels — a legitimate AP should not appear on two channels simultaneously.
5. **Static OUI Database** — New camera models, tracker products, and OUI reassignments require a firmware update. See Section 18.2 for the upgrade path.
6. **Fixed Touch Calibration** — Constants `TX_MIN/TX_MAX/TY_MIN/TY_MAX` are hard-coded for the development unit. Different resistive panels require a startup calibration routine.

22 Version History

v3.1 — Engineering Hardened (Current)

Severity	Change
CRITICAL	Fixed BLE callback data race with FreeRTOS mutex
CRITICAL	Deep sleep now has wakeup source configured on TOUCH_IRQ
HIGH	Replaced all Arduino String heap allocations with char arrays
HIGH	`scaleMultiplier` capped at 4× — prevents energy saturation
HIGH	`earlyActive` reset per scan pass — reduces false positives
HIGH	RF PA level set to MIN for passive receive-only scanning
MEDIUM	Active BLE scan enabled — captures scan response names
MEDIUM	`reflectionSpike` overflow capped at 20
MEDIUM	`totalAlerts` overflow protected at 99999
LOW	Dead extern void declarations removed from loop()
LOW	Magic colour values replaced with named constants
LOW	Extended OUI/tracker database (Tile, Chipolo added)
LOW	`anomalyScore` wired into `anomalyHist` acceleration

Table 12: v3.1 changelog

v3.0 (Previous)

Base version before engineering hardening. Used Arduino String objects throughout. BLE shared data had no mutex protection, causing intermittent data corruption. Deep sleep lacked a wakeup source, causing devices to not wake after timeout. PA level was RF24_PA_MAX, causing SPI self-interference artefacts on the spectrum display.

WatchDog 2.4 — Portable Wireless Security Analyser
Firmware: `Watchdog_Final_V1.ino` · Version v3.1 Engineering Hardened